

UNINASSAU

Disciplina de Compiladores

Mini Compiler

Minilang – Compilador Educacional em Python

Integrantes do Grupo:

Ryan Nunes, Bryan Samuel, Anderson Djalma e Gabriel Barros

Professor: Anderlan

Maceió - AL

30 de maio de 2026

Sumário

1	Introdução	2
2	Estrutura do Compilador	2
2.1	Lexer (Análise Léxica)	2
2.2	Parser (Análise Sintática)	3
2.3	Generator (Geração de Código)	4
3	Sintaxe da Linguagem Minilang	4
3.1	Variáveis e Strings	4
3.2	Operações Matemáticas	4
3.3	Estrutura Condicional	4
3.4	Laço FOR	5
3.5	Laço WHILE	5
4	Tecnologias Utilizadas	5
5	Uso de Inteligência Artificial	5
5.1	Como a IA foi utilizada	5
5.2	Erros encontrados durante o desenvolvimento	6
6	Como Executar o Projeto	6
6.1	Pré-requisitos	6
6.2	Execução	6
7	Conclusão	6

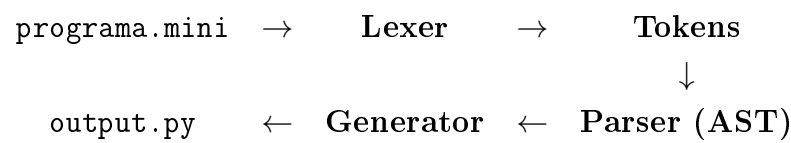
1 Introdução

Este relatório documenta o desenvolvimento do **Mini Compiler**, um compilador educacional criado em Python para a disciplina de Compiladores. O projeto tem como objetivo implementar as etapas fundamentais de um compilador real: análise léxica, análise sintática, geração de Árvore Sintática Abstrata (AST) e geração de código.

A linguagem desenvolvida, chamada **Minilang**, é uma linguagem simples com sintaxe em português/inglês que compila para código Python. Ela suporta variáveis, operações matemáticas, estruturas condicionais e de repetição.

2 Estrutura do Compilador

Um compilador é um programa que traduz código escrito em uma linguagem (linguagem fonte) para outra linguagem (linguagem destino). O Mini Compiler é composto por quatro módulos principais, conforme ilustrado abaixo:



2.1 Lexer (Análise Léxica)

O **Lexer** é a primeira etapa do compilador. Ele lê o código fonte caractere por caractere e agrupa em unidades chamadas **tokens**. Um token é o menor elemento com significado em uma linguagem, como uma palavra reservada, um número ou um operador.

O Lexer do Minilang foi implementado usando a biblioteca **PLY (Python Lex-Yacc)**. Os tokens reconhecidos são:

Exemplo de como o Lexer tokeniza o código:

```
1 LET x = 10
2 IF x > 5
3 PRINT "Maior"
4 END
```

Listing 1: Entrada no Lexer

```
1 LexToken(LET, 'LET', 1, 0)
2 LexToken(ID, 'x', 1, 4)
3 LexToken(EQUALS, '=', 1, 6)
4 LexToken(NUMBER, 10, 1, 8)
5 LexToken(IF, 'IF', 2, 11)
6 LexToken(ID, 'x', 2, 14)
7 LexToken(GREATER, '>', 2, 16)
```

Token	Descrição
LET	Declaração de variável
PRINT	Impressão de valor
IF / ELIF / ELSE	Estrutura condicional
FOR	Laço de repetição com contador
WHILE	Laço de repetição com condição
END	Fechamento de bloco
TO	Intervalo do FOR
NUMBER	Número inteiro ou decimal
STRING	Texto entre aspas
ID	Identificador (nome de variável)
EQUALS / EQUAL_EQUAL	Atribuição (=) e comparação (==)
GREATER / LESS	Operadores > e <
GREATER_EQUAL / LESS_EQUAL	Operadores >= e <=
NOT_EQUAL	Operador !=
PLUS / MINUS / TIMES / DIVIDE	Operadores aritméticos

Tabela 1: Tokens da linguagem Minilang

```

8 LexToken(NUMBER, 5, 2, 18)
9 LexToken(PRINT, 'PRINT', 3, 20)
10 LexToken(STRING, 'Maior', 3, 26)
11 LexToken(END, 'END', 4, 33)

```

Listing 2: Tokens gerados pelo Lexer

2.2 Parser (Análise Sintática)

O **Parser** recebe a lista de tokens do Lexer e verifica se eles formam uma estrutura gramaticalmente válida, de acordo com as regras da linguagem. Ele utiliza o método **LALR (Look-Ahead LR)**, implementado pelo PLY, que é o mesmo método utilizado em compiladores profissionais.

O resultado do Parser é uma **Árvore Sintática Abstrata (AST)**, que representa a estrutura hierárquica do programa.

Exemplo de AST gerada para o código acima:

```

1 [
2   {'type': 'LET', 'name': 'x', 'value': 10},
3   {'type': 'IF',
4     'left': 'x', 'operator': '>', 'right': 5,
5     'body': [{'type': 'PRINT', 'value': 'Maior',
6                 'value_type': 'STRING'}]},
7   'elifs': [], 'else': None}
8 ]

```

Listing 3: AST gerada pelo Parser

2.3 Generator (Geração de Código)

O **Generator** percorre a AST e traduz cada nó para código Python equivalente. Ele utiliza recursão para tratar comandos aninhados (por exemplo, um **LET** dentro de um **IF**), garantindo que a indentação do Python seja gerada corretamente.

```
1 x = 10
2 if x > 5:
3     print("Maior")
```

Listing 4: Código Python gerado

3 Sintaxe da Linguagem Minilang

3.1 Variáveis e Strings

```
1 LET nome = "Brian"
2 LET idade = 22
3 LET soma = idade + 10
```

3.2 Operações Matemáticas

```
1 LET a = 10
2 LET b = 3
3 LET resultado = a * b
4 PRINT resultado
```

3.3 Estrutura Condicional

```
1 LET nota = 7
2
3 IF nota > 7
4     PRINT "Aprovado"
5
6 ELIF nota == 7
7     PRINT "Passou na media"
8
9 ELSE
10    PRINT "Reprovado"
11
12 END
```

3.4 Laço FOR

```
1 FOR i = 1 TO 5
2 PRINT i
3 END
```

3.5 Laço WHILE

```
1 LET x = 1
2 WHILE x < 6
3 PRINT x
4 LET x = x + 1
5 END
```

4 Tecnologias Utilizadas

- **Python 3.13** – Linguagem de implementação do compilador
- **PLY (Python Lex-Yacc)** – Biblioteca para construção do Lexer e Parser LALR
- **Git e GitHub** – Controle de versão e colaboração
- **VS Code** – Ambiente de desenvolvimento
- **MikTeX + TeXMaker** – Compilação e edição deste relatório em $\text{\LaTeX}$

5 Uso de Inteligência Artificial

Conforme permitido pelas regras da AV2, o grupo utilizou Inteligência Artificial (Claude – Anthropic) como apoio no desenvolvimento do projeto. A seguir estão os principais usos e os erros encontrados no processo.

5.1 Como a IA foi utilizada

- Revisão e melhoria do `generator.py`
- Correção de erros de importação no Python
- Criação do arquivo `programa.mini` com exemplos completos
- Colaboração deste relatório em $\text{\LaTeX}$

5.2 Erros encontrados durante o desenvolvimento

1. **ImportError: cannot import name 'lexer' from 'lexer'**

O arquivo `lexer.py` estava incompleto – faltava a linha `lexer = lex.lex()` no final, que cria e exporta o objeto do lexer.

2. **ImportError: cannot import name 'parser' from 'parser'**

O Python possui um módulo nativo chamado `parser`, que conflitava com o arquivo `parser.py` do projeto. A solução foi renomear o arquivo para `minilang_parser.py` e atualizar os imports.

6 Como Executar o Projeto

6.1 Pré-requisitos

```
pip install ply
```

6.2 Execução

1. Escreva o código Minilang no arquivo `programa.mini`

2. Execute o compilador:

```
python main.py
```

3. O código Python gerado será salvo em `output.py`

4. Para executar o resultado:

```
python output.py
```

7 Conclusão

O desenvolvimento do Mini Compiler permitiu ao grupo compreender na prática as etapas fundamentais de um compilador: análise léxica, análise sintática, construção da AST e geração de código. A utilização da biblioteca PLY facilitou a implementação do Lexer e do Parser LALR, enquanto a refatoração do Generator com recursão garantiu suporte a estruturas aninhadas.

O projeto atingiu todos os requisitos da AV2: possui condicional (IF/ELIF/ELSE), laços de repetição (FOR e WHILE), versionamento no GitHub, e este relatório elaborado no editor tipográfico L^AT_EX.